Operating Systems: Test 2

Threads · Scheduling · Mutual Exclusion · Concurrency Problems · Deadlock · File Management. One letter sheet both sides.

Deadlock detection (guaranteed question)

Uses **Request Q** (current requests), **Allocation A**, **Available V**. No claim matrix.

1. **Mark** every process whose **A row is all 0** (holds nothing, can't be in a deadlock).
2. **W ← V**.
3. Find **unmarked P_i with $Q_i \le W$** (every column). None? **stop**.
4. Mark P_i; **W ← W + A_i**; go to 3.

Deadlock ⟺ unmarked processes remain; the unmarked ones are deadlocked. Pick order doesn't matter.

```
Fig 6.10: R=(2,1,1,2,1) V=(0,0,0,0,1)
   Q          A
P1 0 1 0 0 1  1 0 1 1 0
P2 0 0 1 0 1  1 1 0 0 0
P3 0 0 0 0 1  0 0 0 1 0
P4 1 0 1 0 1  0 0 0 0 0
mark P4 (A=0); W=(0,0,0,0,1)
P3 fits → mark, W=(0,0,0,1,1)
P1 needs R2, P2 needs R3 → stop
P1, P2 deadlocked (P1→R2→P2→R3→P1)
```

Resource allocation graph

- Circle per process; box per resource with **one dot per instance** (count from R).
- **Request edge** P → R (asked, not granted): one per unit in Q.
- **Assignment edge** R's dot → P (granted): one per unit in A.
- **Cycle + single instances = deadlock.** Multiple instances: a cycle may *not* be a deadlock (6.5d); confirm with the algorithm.
- RAG detection scales badly (node per instance) and handles divisible resources (memory) poorly.

Banker's algorithm (avoidance)

R (resources), V (available), C (claim = max), A (alloc). **Need = C − A**. $R_j = V_j + \Sigma_i A_{ij}$.

Safety test: avail ← V; repeat: find P_k with $C_k - A_k \le$ avail; avail += A_k, remove P_k. **Safe ⟺ everyone removed.** Final avail = R (sanity check).

Request: alloc+req > claim → error; req > avail → suspend; else *pretend* to grant, run safety: safe → grant, unsafe → restore + suspend.

```
Handout: V=(0,1,1); need P1(2,2,2)
P2(0,0,1) P3(1,0,3) P4(4,2,0)
P2→(6,2,3) P1→(7,2,3) P3→(9,3,4) P4→(9,3,6)
safe
Fig 6.8: P1 gets (1,0,1) → V=(0,1,1), all
need R1 → unsafe
```

Safe = some order lets all finish. **Unsafe ≠ deadlocked** (only might be). Restrictions: max claims known in advance, independent processes, fixed # resources, no exit while holding.

File-management math

Indexed seq.	`no index: N/2`
	`index of k entries: k/2 + N/(2k)`
	$N=10^5$, k=100: 50+500 = **550** (vs 50,000)
	$N=2\cdot10^6$, k=500: 250+2000 = **2250**
	best k=√N → √N total (N=40,000: k=200, **200**)

Bit table	`blocks = disk / block size`
	`bits = blocks; bytes = blocks/8`
	16 GB, 4 KB: $2^{34}/2^{12}=2^{22}$ bits = **512 KB**
	1 TB, 4 KB: 2^{28} bits = 2^{25} B = **32 MB**

Free block list	`blocks × (24 or 32 bits) → 24–32× bit table` 16 GB, 4 KB, 32-bit: $2^{22}\times4$ B = **16 MB** (32×)

Bit reading	`0 = free, 1 = used. Count 0s;`
	`first 0 = first free;`
	`run of k free = k consecutive 0s.`
	`1101 0011 1000 1011 → free 2,4,5,9,10,11,13 (7); run of 3 = 9–11`

Blocking	`fixed: ⌊B/R⌋ per block, waste B − n·R` B=1024, R=100: **10**/block, **24 B** wasted
Idx portions	`walk portions summing lengths: (1,3)(28,4)(14,1)` `file blk 5 = 2nd of (28,4) → disk` **29**
Compaction	`pack in disk order: start = prev start + prev length`

I · THREADS

- Process = **resource ownership** (address space, protection) + **scheduling/execution** (state, priority). Separable.
- **Thread** (lightweight process) = unit of **dispatching**. **Process** (task) = unit of **resource ownership**, "unit of resource allocation and protection".
- **Multithreading**: many concurrent paths in one process. Single-threaded approach: thread not recognized. Java runtime = 1 process, many threads.
- Per thread: state, saved context, **execution stack**, per-thread static storage, **TCB**, user + kernel stack. Shared: **PCB**, address space, resources.
- Benefits: faster **create**, **terminate**, **switch**, cheaper **communication** (shared memory, no kernel).
- Uses: foreground/background, asynchronous processing, speedup, modular structure.
- Scheduled per thread. **Suspend/terminate a process ⟹ all its threads.**
- States: Running, Ready, Blocked (**no suspend**: suspend is a process concept). Ops: **Spawn, Block, Unblock, Finish** (finish frees register context + stacks).

II · SCHEDULING

Type	Decides
Long-term	admit to pool (degree of multiprogramming); New→Ready
Medium-term	swap in/out (suspend states)
Short-term	dispatcher: which ready process runs; on clock/I/O interrupts, OS calls, signals
I/O	which pending I/O request a device handles

Criteria. User/perf: turnaround, response time, deadlines. User/non-perf: predictability. System/perf: throughput, utilization. System/non-perf: fairness, priorities, balancing resources.

Pure priority starves low priorities; fix = **aging** (priority rises with wait). **Preemptive**: more overhead (context switches), better service (no monopoly).

`w = waiting so far, e = executed so far, s = total service`
T_r `= finish − arrival; normalized =` T_r/T_s
`HRRN: R = (w + s)/s`

Policy	Select	Mode	Notes
FCFS	max w	non	bad for short, favors CPU-bound
RR	const	pre (q)	time slicing; q slightly > typical interaction; I/O-bound shortchanged → **virtual RR**
SPN	min s	non	needs s estimates; long jobs starve; less predictable; bad for time-sharing
SRT	min s−e	pre (arrival)	best mean T_r; starvation; no clock interrupts
HRRN	max R	non	aging, no starvation
FB	queue lvl	pre (q)	no s needed; enter RQ0, demote per preemption; bottom FCFS; long jobs stretch (q=2^i helps)

`Example A–E arr 0,2,4,6,8 serv 3,6,4,5,2 (mean` T_r, T_r/T_s`):`
`FCFS 8.6/2.56 RR1 10.8/2.71 RR4 10.0/2.71 SPN 7.6/1.84`

`SRT 7.2/1.59` `HRRN 8.0/2.14 FB1 10.0/2.29 FB2`^{i} `10.6/2.63`

Windows: highest priority wins, preempts lower; 16–31 real-time, 1–15 variable, 0 zero-page thread. Base = class + level: Normal class → below 7, normal 8, above **9**; High/normal 13. RR within a level. Boosts (input, finished waits) drop 1 level per full quantum; none for 16–31. Starvation **mitigated, not eliminated** (real-time can hog). Priority + boosts vs Linux weights. **Affinity**: allowed CPUs; **load balancing**: spread work. **Linux**: SCHED_OTHER (fair), FIFO/RR (real-time), DEADLINE (EDF). Lower nice = bigger weight; share = $w_i/\Sigma w$. **CFS**: min virtual runtime, $\Delta v = \Delta t \cdot w_0/w$ (heavy task's v grows slower → picked more). **Lag** = service owed (+ = under-served). **EEVDF** (6.6): eligible (lag ≥ 0) with earliest d = v + r·w_0/w.

III · MUTUAL EXCLUSION

- **Race condition**: shared data, ≥1 writer, result depends on timing. **Critical section**: code using shared resource. **Mutual exclusion**: only one in its CS at a time. **Atomic**: indivisible.
- **Livelock**: keep changing state, no progress. **Starvation**: runnable but never chosen. **Busy waiting** = spin testing a variable.
- Competing processes: mutual exclusion, deadlock, starvation. Concurrency ⊇ parallelism. Memory arbiter serializes access, no fairness.
- Multiprogramming (1 CPU) / multiprocessing (multi CPU) / distributed processing.

Attempt	ME?	Problem
1 turn var	yes	strict alternation; failure blocks other
2 check then set flag	**no**	both pass check
3 set flag then check	yes	**deadlock**
4 set, defer, reset	yes	**livelock**
Dekker	yes	flags + turn breaks tie
Peterson	yes	simpler

`Peterson P0: flag[0]=true; turn=1; while(flag[1] && turn==1); CS; flag[0]=false;`

Interrupt disabling: works on uniprocessor only; hurts efficiency. **Compare&swap** (atomic; returns old): `while(cas(bolt,0,1)==1); CS; bolt=0;` + any # processes, uni/multi, many CSs; − busy wait, starvation, deadlock.

Semaphore: init ≥0; `semWait: count--; if <0 block`; `semSignal: count++; if ≤0 unblock one`. Negative count = # waiting. Can't read it. **Binary**: 0/1. **Strong** = FIFO (no starvation), **weak** = unspecified. Must be atomic (hardware, Dekker/Peterson, CAS, interrupts). **Mutex**: locker must unlock (binary sem: anyone may signal).

Monitor: local data only via its procedures; enter by calling one; **one process inside at a time**. `cwait(c)` blocks + frees monitor; `csignal(c)` resumes one; **broadcast** resumes all. **Signals not saved** (a semaphore count would remember).

IV · CONCURRENCY PROBLEMS

Dining philosophers: 5 forks, need 2. Naive (sem per fork, left then right) → **deadlock** if all take left. Fix: 5 more forks, or `room = 4` (wait(room) before forks, signal after). Monitor: cwait on `ForkReady[i]` per missing fork.

Producer/consumer: one at a time in buffer; no add when full, no take when empty.

- Binary sems s=1, delay=0 **wrong**: consumer tests `n==0` outside CS; stale delay signal → **n = −1**. Fix: `m = n` inside CS, test m.
- Counting: `n=0, s=1`. Prod: `wait(s) append signal(s) signal(n)`. Cons: `wait(n) wait(s) take signal(s) consume`.
- Swap producer signals: **OK**. Swap consumer `wait(n)/wait(s)`: **deadlock**.
- Bounded: add `e = size`; prod `wait(e)` first; cons `signal(e)` after take.

Readers/writers: many readers OR one writer. **Readers priority**: wsem guards data, read_lock guards readcount; first reader waits wsem, last signals → **writers starve**. **Writers priority**: a waiting writer blocks new readers (readblock/rsem; writepending/z lets 1 reader queue on rsem) → **readers starve**. Neither: FIFO queue via semaphore.

V · DEADLOCK

Deadlock: permanent blocking of a set, each waiting on an event only another in the set can cause. **Reusable** (CPU, I/O channels, memory, devices, files, semaphores) vs **consumable** (interrupts, signals, messages; e.g. blocking Receive before Send). **Fatal region** in a joint progress diagram.

Condition	Prevent by
Mutual exclusion	generally can't
Hold & wait	request all at once (long waits, idle resources, foreknowledge)
No preemption	release on denial / OS preempts holder
Circular wait	linear ordering of resource types

First 3 necessary, not sufficient; all 4 = necessary + sufficient. Prevention: **indirect** (first 3) / **direct** (circular wait). **Avoidance**: process initiation denial or resource allocation denial (banker's). Avoidance +: no preempt/rollback, less restrictive than prevention.

Detection: grant freely, check periodically (per request: early + simple, but costs CPU).

Memory 200 KB: P1 80 then 60, P2 70 then 80 → after 150, 50 left, neither 2nd request fits → deadlock. Receive(P2) then Send vs Receive(P1) then Send: blocking receives → deadlock.

Recovery: abort all; back up to checkpoint + restart; abort one by one; preempt one by one. **Integrated**: classes ordered; swappable space = all at once; process resources = ordering/avoidance; main memory = preemption; internal = ordering. Linux/Windows mostly ignore deadlock.

VI · FILE MANAGEMENT

File properties: long-term existence, sharable, structure. Field → record → file → database. Modern OSs: file = **byte stream**. Layers: access methods → logical I/O (records) → basic I/O supervisor → basic file system (blocks) → device drivers.

Org	Key facts
Pile	arrival order; exhaustive search; easy append
Sequential	sorted by key; batch/tape; inserts to log file + batch update
Indexed seq.	+ index + overflow file; random access, still sequential
Indexed	only via indexes; exhaustive/partial; airline, inventory
Hashed	key → bucket; fast exact; overflow chains; bad range queries

Blocking: fixed (internal frag) · variable spanned (no waste, pointer) · variable unspanned (end waste).

	Contig.	Chained	Idx blk	Idx var
Prealloc	necessary	possible	possible	
Portions	variable	fixed	fixed	variable
Size	large	small	small	medium
Alloc freq	once	low–high	high	low
Time	medium	long	short	medium
FAT size	1 entry	1 entry	large	medium

Contiguous: external frag → compaction. Chained: pointer per block, no ext frag, sequential, no locality → consolidation. Indexed: index block, sequential + direct.

Free space (disk allocation table): **bit table** (bit/block, 0 free, any method, smallest) · **chained free portions** (no table; fragments; read before alloc) · **indexing** (free space as a file) · **free block list** (24/32-bit numbers on disk; stack or FIFO cached).

VII · KEY CODE

Semaphore primitives (Fig 5.6 / 5.7)

```
semWait(s): s.count--;
  if (s.count < 0) { put in s.queue; block;
}
semSignal(s): s.count++;
  if (s.count <= 0) { move P from s.queue
to ready; }
semWaitB(s): if (s.value == 1) s.value = 0;
  else { put in s.queue; block; }
semSignalB(s): if (s.queue empty) s.value =
1;
  else { move P from s.queue to ready; }
```

Atomic via CAS: spin on `cas(s.flag,0,1)` only around the primitive's few lines (not the whole CS). Via interrupts: inhibit, update, allow.

Dekker (P0; P1 symmetric) · init turn=1

```
flag[0] = true;
while (flag[1])
  if (turn == 1) {
    flag[0] = false;
    while (turn == 1) ;  /* wait */
    flag[0] = true; }
/* critical section */
turn = 1; flag[0] = false;
```

Only the process whose turn it *isn't* backs off → no livelock.

Bounded buffer (Fig 5.16) · s=1, n=0, e=size

```
producer: produce(); semWait(e);
semWait(s);
  append(); semSignal(s); semSignal(n);
consumer: semWait(n); semWait(s); take();
  semSignal(s); semSignal(e); consume();
```

s = mutual exclusion on buffer; n = filled slots (consumer waits if empty); e = empty slots (producer waits if full).

Incorrect P/C (Fig 5.12) · s=1, delay=0, n=0

```
producer: produce(); semWaitB(s); append();
n++;
  if (n==1) semSignalB(delay);
semSignalB(s);
consumer: semWaitB(delay); loop:
semWaitB(s); take();
  n--; semSignalB(s); consume();
  if (n==0) semWaitB(delay);
```

Readers priority (handout Alg 1)

```
READER: wait(read_lock); readcount++;
  if (readcount==1) wait(wsem);
  signal(read_lock); READUNIT();
  wait(read_lock); readcount--;
  if (readcount==0) signal(wsem);
  signal(read_lock);
WRITER: wait(wsem); WRITEUNIT();
signal(wsem);
```

Writers priority (handout Alg 2)

```
readblock=writeblock=writepending=1;
mutexes r_, w_
READER: wait(writepending);
wait(readblock);
  r.lock; readcount++; if ==1
wait(writeblock); r.unlock;
  signal(readblock); signal(writepending);
READUNIT();
  r.lock; readcount--; if ==0
signal(writeblock); r.unlock;
WRITER: w.lock; writecount++; if ==1
wait(readblock); w.unlock;
  wait(writeblock); WRITEUNIT();
signal(writeblock);
  w.lock; writecount--; if ==0
signal(readblock); w.unlock;
```

Stallings names: readblock = **rsem**, writepending = **z**. Readers only: wsem set, no queues. Writers only: wsem + rsem set, writers queue on wsem. Mixed: one reader queues on rsem, others on z, writers on wsem.

Dining philosophers with room (deadlock-free)

```
semaphore fork[5] = {1}, room = 4;
think(); wait(room); wait(fork[i]);
wait(fork[(i+1) mod 5]); eat();
signal(fork[(i+1) mod 5]); signal(fork[i]);
signal(room);
```

Monitor version: `get_forks(pid)`: left=pid, right= (pid+1)%5; if fork not free, `cwait(ForkReady[f])`; take. `release_forks`: if `empty(ForkReady[f])` mark free, else `csignal`.

VIII · WORKED SCHEDULING

```
A0/3 B2/6 C4/4 D6/5 E8/2 (arrival/service)
FCFS: A0-3 B3-9 C9-13 D13-18 E18-20
  Tr 3,7,9,12,12 → 8.60
SPN: A0-3 B3-9 E9-11 C11-15 D15-20
  Tr 3,7,11,14,3 → 7.60
SRT: finish A3 C8 E10 B15 D20
  Tr 3,13,4,14,2 → 7.20
HRRN: A0-3 B3-9; t=9: C R=(5+4)/4=2.25,
  D (3+5)/5=1.6, E (1+2)/2=1.5 → C9-13;
  t=13: D (7+5)/5=2.4, E (5+2)/2=3.5 → E13-
15, D15-20
  Tr 3,7,9,14,7 → 8.00
```

Windows (q=4, A,B pri 8; C pri 10 arrives at 6): A 0–4, B 4–6, C preempts 6–8. **CFS** (w_0=1; A w2 v10, B w1 v8, C w4 v9): pick B; 3 ms → v=11; next C (9); C runs 4 ms → +4/4 = 10. **EEVDF** V=13: A v12 r4 w2 → d14; B v10 r6 → d16; C v15 > V *ineligible* → **A**. **Weights** 3,1,1,3 → 3/8,1/8,1/8,3/8 (40 ms: 15,5,5,15); D blocks → 3/5, 1/5, 1/5.

IX · COMMONLY CONFUSED

- **Process vs thread.** Process owns resources + protection; thread is what gets dispatched.
- **Deadlock vs livelock vs starvation.** Blocked forever / busy but no progress / runnable but never picked.
- **Prevention vs avoidance.** Prevention removes a condition by design; avoidance checks each request dynamically (safe state).
- **Avoidance vs detection.** Claim matrix C, every request / request matrix Q, periodic + recovery.
- **Unsafe vs deadlocked.** Unsafe = no guarantee; not yet deadlocked.
- **Cycle in RAG.** Deadlock only certain with single instances.
- **Binary semaphore vs mutex.** Anyone may signal a binary semaphore; only the locker unlocks a mutex.
- **Strong vs weak semaphore.** FIFO (starvation-free) vs unspecified order.
- **Semaphore vs condition variable.** Count remembers a signal; csignal with no waiter is lost.
- **SPN vs SRT.** Same idea; SRT preempts on arrival using remaining time.
- **Concurrency vs parallelism.** Overlap/interleaving vs truly simultaneous.
- **Indexed sequential vs indexed.** Keeps sequential order + overflow file vs access only through indexes.
- **Chained alloc vs chained free portions.** File's blocks linked vs free space linked.
- **Compaction vs consolidation.** Contiguous (squeeze out holes) vs chained/indexed (bring a file's blocks together).
- **Preallocation vs dynamic.** Max size at creation vs portions as needed.
- **Lost vs wasted wakeup.** Signal missed (waiter keeps sleeping) vs woken but must wait again.
- **Reusable vs consumable.** Not depleted by use vs created and destroyed.

X · WAITING CORRECTLY (ASSIGNMENT 2)

- **Lost wakeup**: a signal that should resume a waiter is missed (wrong thread woken, or no one waiting yet). One shared "not empty" condition + single signal for two kinds of consumers can wake the wrong one.
- **Wait in a `while`, not `if`**: state may change again (or spurious wakeup), so re-check the predicate. This is **not busy waiting** (the thread blocks).
- **TOCTOU** (time of check to time of use): check and act in separate critical sections → state changes in between. Fix: check + act under one lock.
- **Targeted blocking**: each waiter blocks on the specific condition it needs (separate semaphores/conditions per kind + space).
- **Wasted wakeup**: resumed only to wait again. **Broadcast** to all waiters is correct (nobody eligible left asleep) but wasteful.

XI · MORE FILE MANAGEMENT

Objectives: meet user data needs, valid data, performance, many device types, minimize lost data, standard I/O routines, multi-user support. **User requirements**: create/delete/read/write/modify; controlled access to others' files and control of own; move data between files; back up/recover; access by name.

Criteria for an organization: short access time, ease of update, economy of storage, simple maintenance, reliability.

Indexed seq. lookup: index → highest key ≤ target → pointer → scan main file. Multi-level index for big files.

Portion size: large variable (fast, small tables) vs small fixed blocks (flexible, big tables).